

The solution:

State templating

with `{var}`

Reference: [ADK Documentation - State Templating](#)

From ADK documentation

“To inject a value from the session state, enclose the key of the desired state variable within curly braces: `{key}`. The framework will automatically replace this placeholder with the corresponding value from `session.state` before passing the instruction to the LLM.”

From ADK documentation

“The `{var}` syntax is a placeholder. Before the instruction is sent to the LLM, the ADK framework automatically replaces (Example: `{topic}`) with the value of `session.state['topic']`. This is the recommended way to provide context to an agent, using templating in the instructions.”



How it works

Python

```
# State contains user information
session.state["user_name"] = "Alex"

# Agent uses templating
agent = LlmAgent(
    instruction="Hello {user_name},
    how can I help you today?"
)

# Before LLM sees the instruction,
ADK automatically:
# 1. Looks up
session.state["user_name"]
# 2. Replaces {user_name} with "Alex"
# 3. Sends resolved instruction to
LLM: "Hello Alex, how can I help you
today?"
```

Complete example

Python

```
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

# Setup
session_service = InMemorySessionService()

session = session_service.create_session(app_name="app", user_id="user1")

# Manually set state (simulating
previous interaction or external
data)
session.state["user_name"] = "Alex"
session.state["user_language"] = "Spanish"

# Agent with templating
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="Hello {user_name}.
    Please respond in {user_language}."
)

# When agent runs:
# LLM receives: "Hello Alex. Please
respond in Spanish."
```


Core concepts

1. State templating syntax: `{key}`

Reference: [ADK documentation - Using {key} templating](#)

From ADK documentation

“To inject a value from the session state, enclose the key of the desired state variable within curly braces: `{key}`. The framework will automatically replace this placeholder with the corresponding value from `session.state` before passing the instruction to the LLM.”

Basic syntax

```
Python
instruction="Process {variable_name}"

# ADK automatically:
# 1. Scans instruction for {key}
# patterns
# 2. Looks up
# session.state["variable_name"]
# 3. Replaces {variable_name} with
# actual value
# 4. Sends resolved instruction to LLM
```

Example

```
Python
# State contains:
session.state["topic"] = "quantum
computing"

agent = LlmAgent(
    instruction="Provide an overview
of {topic}"
)

# Before LLM call, instruction becomes:
# "Provide an overview of quantum
computing"
```



Key points

- 1 Templating happens before each agent invocation
- 2 Variables are replaced with their current state values
- 3 Works with any state key
- 4 Case-sensitive: `{name}` $\neq$ `{name}`

2. Optional templating: `{key?}`

Reference: [ADK Documentation - Important Considerations](#)

From ADK documentation

“Ensure that the key you reference in the instruction string exists in the session.state. If the key is missing, the agent will throw an error. To use a key that may or may not be present, you can include a question mark (?) after the key (e.g. `{topic?}`).”

Why you need it

```
Python
# ❌ Without optional syntax - errors
# if key missing
instruction="Greet {user_name}"
# If state["user_name"] doesn't exist
# → Error!

# ✅ With optional syntax - no error
# if key missing
instruction="Greet {user_name?}"
# If state["user_name"] doesn't exist
# → resolves to "Greet "
```

Syntax variations

Python

```
# Basic optional - empty if missing
"{var?}"
# Result if var missing: "" (empty string)

# Optional with default text
"{var?default text}"
# Result if var="value": "value"
# Result if var missing: "default text"

# Conditional text block
"{premium_user?You have premium access.}"
# Result if premium_user exists and truthy: "You have premium access."
# Result if premium_user missing or falsy: "" (empty string)
```

Practical example

Python

```
agent = LlmAgent(
    instruction="""
    You are a helpful assistant.

    User preferences:
    - Name: {user_name?Guest}
    - Language: {user_language?English}

    {premium_user?✨ Premium features are enabled.}

    Respond to the user's question.
    """
)

# If user_name not in state, defaults to "Guest"
# If premium_user not in state, that line disappears entirely
```

3. How templating works with state

Pattern

None

graph LR

```
    A[State contains<br/>
    >user_name='Alex'] -->|Templating|
    B[Instruction:<br/>'Hello user_name']
    B -->|ADK resolves| C[LLM
    receives:<br/>'Hello Alex']
```

```
    style A fill:#e1f5ff
```

```
    style C fill:#eeffee
```

Execution flow

Python

```
# 1. State is set (from previous turn or manually)
```

```
session.state["user_name"] = "Alex"
session.state["topic"] = "quantum computing"
```

```
# 2. Agent has templated instruction
agent = LlmAgent(
```

```
    instruction="Hello {user_name}.
    Let's discuss {topic}."
)
```

```
# 3. When agent runs, ADK resolves templates:
```

```
# Before: "Hello {user_name}. Let's discuss {topic}."
```

```
# After: "Hello Alex. Let's discuss quantum computing."
```

```
# 4. LLM receives resolved instruction
```

```
# 5. LLM generates response based on resolved instruction
```

Key insight

Templating makes instructions dynamic based on state, without changing agent code.

Hands-on example

Building a personalized greeting agent

What you'll build: An agent that uses state templating to create personalized responses based on user information stored in state.

Step 1: Create the project

Shell

```
adk create personalized_greeter
cd personalized_greeter
```

Step 2: Write the agent

Replace the contents of `agent.py` with:

Python

```
"""
```

Personalized Greeter - Demonstrates
State Templating
Shows how {var} templating injects
state values into instructions.

Reference: <https://google.github.io/adk-docs/sessions/state.md>
"""

```
from google.adk.agents import LlmAgent
```

```
# Agent with state templating
```

```
root_agent = LlmAgent(  
    model='gemini-2.5-flash',  
    name='personalized_greeter',
```

```
    instruction="""  
    You are a friendly assistant.
```

```
    User information:
```

```
    - Name: {user_name?there}
```

```
    - Preferred language:
```

```
{user_language?English}
```

```
    - Membership: {membership_tier?  
free}
```

```
    {membership_tier?Your membership  
level is: {membership_tier}}
```

```
    Greet the user warmly and offer  
assistance.
```

```
    Respond in {user_language?  
English}.
```

```
    """
```

```
)
```

Code walkthrough

Lines 13–24: Templated instruction

- `user_name?there` - Uses “there” if `user_name` not in state
- `{user_language?English}` - Defaults to “English” if not set
- `{membership_tier?free}` - Shows “free” if not specified
- Conditional line only appears if `membership_tier` exists
- Instruction adapts based on available state

Step 3: Test with runner

Create `test_templating.py`:

Python

"""

Test state templating with different state values.

Run with: `python test_templating.py`

"""

```
from agent import root_agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part
```

Setup

```
session_service = InMemorySessionService()
session = session_service.create_session(
    app_name="greeter_app",
    user_id="user1",
    session_id="session1"
)
```

```
runner = Runner(
    agent=root_agent,
    app_name="greeter_app",
    session_service=session_service
)
```

```
# Test 1: No state set (all defaults)
print("=== Test 1: No state (all defaults) ===")
```

```
result1 = runner.run(
    user_id="user1",
    session_id="session1",
    new_message=Content(parts=[Part(text="Hello")])
)
```

```
for event in result1:
    if event.is_final_response():
        print(f"Agent: {event.content.parts[0].text}\n")
```

Test 2: Set user name only

```
print("=== Test 2: With user name ===")
```

```
session.state["user_name"] = "Alex"
```

```
result2 = runner.run(
    user_id="user1",
    session_id="session1",
    new_message=Content(parts=[Part(text="Hello again")])
)
```

```
for event in result2:
    if event.is_final_response():
        print(f"Agent: {event.content.parts[0].text}\n")
```

Test 3: Set all state values

```
print("=== Test 3: With all state values ===")
```

```
session.state["user_name"] = "Alex"
session.state["user_language"] = "Spanish"
session.state["membership_tier"] = "premium"
```

```
result3 = runner.run(
    user_id="user1",
    session_id="session1",
    new_message=Content(parts=[Part(text="Hola de nuevo")])
)
```



```
for event in result3:
    if event.is_final_response():
        print(f"Agent:
{event.content.parts[0].text}\n")

print("=== Current state ===")
print(session.state)
```

Run it:

```
Shell
python test_state.py
```

Expected output:

```
None
=== Test 1: No state (all defaults)
===
Agent: Hello there! I'm here to
help you with anything you need.
How can I assist you today?

=== Test 2: With user name ===
Agent: Hello Alex! Great to see you
again. How can I help you today?

=== Test 3: With all state values
===
Agent: ¡Hola Alex! Tu nivel de
membresía es: premium. ¿En qué
puedo ayudarte hoy?

=== Current state ===
{'user_name': 'Alex',
'user_language': 'Spanish',
'membership_tier': 'premium'}
```

What to notice

- ✓ Test 1: All defaults used (`{user_name?there}` → “there”)
- ✓ Test 2: Partial state (`{user_name}` → “Alex”, others default)
- ✓ Test 3: Full state, Spanish response, membership shown
- ✓ Instruction adapts dynamically based on available state

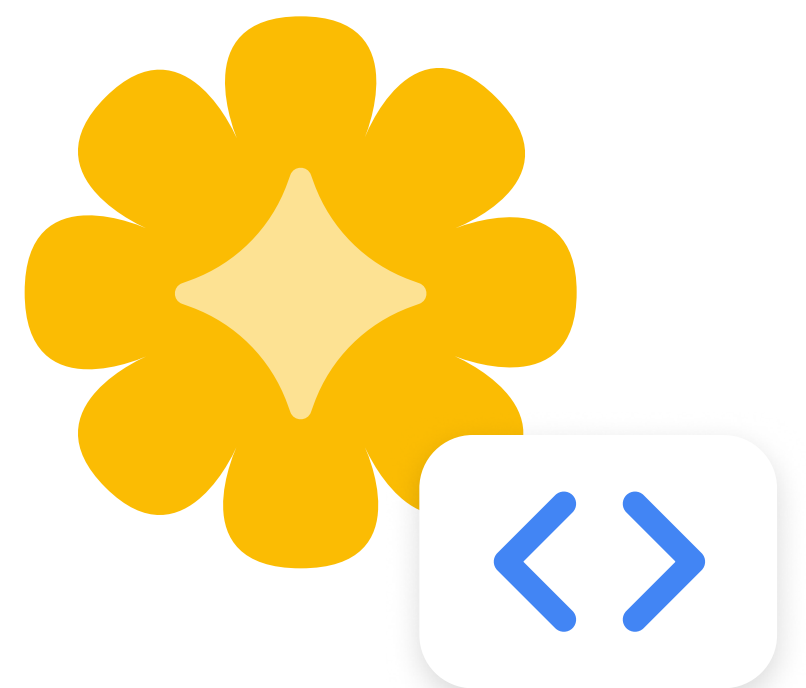
Key takeaways

State templating enables dynamic instructions:

- Use `{var}` syntax to inject state values into instructions
- ADK resolves templates **before** sending to LLM
- Works with any state key
- Makes agents context-aware without code changes

Optional syntax prevents errors:

- `{var}` → Errors if missing (strict mode)
- `{var?}` → Empty if missing (optional mode)
- `{var?default}` → Uses default text if missing
- `{var?Conditional text}` → Shows text only if var exists



Common pattern:

Python

```
# 1. State is set (from previous turn  
or external data)
```

```
session.state["user_name"] = "Alex"
```

```
# 2. Agent uses templating
```

```
agent = LlmAgent(  
    instruction="Hello {user_name}?  
there}"  
)
```

```
# 3. Instruction resolves  
automatically
```

```
# LLM receives: "Hello Alex"
```

Next part

You'll learn about state namespaces (`temp:`, `user:`, `app:`) that control how long state values persist.

When to use templating

- ✓ When instructions need to adapt based on state
- ✓ When personalizing agent behavior
- ✓ When injecting exact values from your application
- ✓ When making instructions context-aware

